MacroSim: Recursive Endogenous Simulation Library for Macroeconomic Models

Güney Kıymaç

August 7, 2025

Contents

1 Introduction

Macroeconomic simulation is a powerful, common, and well-studied tool widely used in policy testing/analysis, forecasting, and understanding the dynamics of economies. With parametric equations, differential systems, and stochastic processes, the field is developed and standardized to a great extent. However, as opposed to natural sciences, economies are not universally explained through concrete theoretical frameworks. This has led to a unique pursuit of generalization in economic contexts, where parametric and weighted adjustments became inevitable for accurate modeling.

MacroSim is a library that came to life through a thought experiment which started as "How economics would have evolved if economic interactions were modeled with some individuality, not just as parts of a system?". This evolved into the thought of a process where variables are not inferred through mathematical transformations of core equations, but are modeled from the ground up from the components of the equations. Challenging the current theory is not the goal of this thought and the library that resulted from it. MacroSim was made as a toolkit for people who feel some benefit may come from exploring this approach.

Through equation discovery aided by AR processes, MacroSim creates a simulation space through individually modeled features. Importantly, no parametric or template-based equations are utilized beyond the use of linear autoregression. This creates a powerful yet brittle discovery space, and it is certainly not recommended where accuracy is the primary concern. Interesting/promising relationships uncovered by this process can be analyzed and tested in more robust and conventional environments.

2 Module Descriptions

MacroSim Modules		
Module	**Description**	**User Accessible**
`SeriesAccessor`	Collects data from FRED and formats it into the expected structure.	**YES**
`AutoReg`	Prepares the data, selects lags and features, and runs autoregression based symbolic equation discovery.	**YES**
`AutoReg._Predictor`	Contains a single fitted expression ready for prediction. Outputted from `AutoReg`.	**YES**
`AutoReg._BatchedPredictor`	Contains multiple fitted expressions ready for prediction. Outputted from `AutoReg`.	**YES**
`stats.Causality`	Automatically computes causality tests and attaches the results to the `DataFrame`.	**NO**
`stats.Stationarity`	Automatically computes stationarity tests and attaches the results to the `DataFrame`.	**NO**
`_batch_fit`	Script for parallel execution of ensemble learning. Called during fit-time of a `BatchedPredictor`.	**NO**

3 Modeling Methodology

3.1 Overview

MacroSim bases its discovery process on top of a simple $AR(n)$ process where n defines the number of lags for the target variable. The constant c and error term ϵ are assumed 0 and inclusion of noise and linear offest is offloaded to the discovery engine.

To aid discovery, supplementary variables are selected from the feature space through the use of Granger Causality Tests (GCT) and Augmented Dickey–Fuller Tests (ADF). This creates a set of variables that are either direct lags of the target, or are both stationary and have a Granger causal relationship with the target.

A symbolic regressor (SR) is responsible for patching these variables in a meaningful way while fitting the AR weights simultaneously. Denoting the target as y and the supplementary features as X_i, the ultimate resulting expression follows the form $f(AR_y(n), X_1, \ldots, X_i)$. This process applied to all variables in the feature space to produce a complete and self-sustaining simulation model.

3.2 Data Preparation

3.2.1 Collection and Formatting

MacroSim requires a dataset of time series with matching dates and frequencies. There are methods implemented to get, re-index, impute, and format data directly through methods supplied in the library[1]. The data is expected to be in a `pandas.DataFrame` with a `DatetimeIndex` and is expected to be entirely numeric. There are no expected encoding specifications for categorical variables, but symbolic regression may yield better results with One-hot Encoding due to the possibility of having individual coefficients for each category.

3.2.2 Splitting

The target variable is passed to the `AutoReg` class as a string. The processing steps of splitting the features, and creating train/test splits are handled internally. `AutoReg` contains methods to batch and shuffle the data to prevent overfitting due to ordered data and utilize an ensemble learning technique for further robustness.

3.2.3 Inter-Process Data Transmission

Due to the limitations with the dependencies of `MacroSim`, creating thread pools with shared memory is not possible. To allow true parallelism, `_batch_fit` and `AutoReg` communicate by stripping the non-serializable portions of the model and its output. This effectively destroys all but the fitted equation as a string and some limited metadata. The

[1] See subsection 4.2

equations are converted to `sympy.Expr` objects and serialized through `pickle`. Upon loading the equations, the metadata is used to reconstruct the model as a native python callable through `sympy.lambdify`.

`_Predictor` and `_BatchedPredictor` classes wrap these callables and define the standard methods expected to exist in a fitted model. (mainly `predict` and some scoring functions)

3.3 Target Lag Selection

By default, lags of the target feature are configured to cover a 2-year period of observations. (two complete cycles of seasonality in many economic contexts) This is adjustable through the `AutoReg.n_lags`[1] attribute, which will be automatically populated with the default lag count using the formula $n_{lags} = 2s$ where s denotes the inferred yearly observation frequency.

Importantly, lag generation causes data to be shifted, which results in a loss of the first n_{lags} observations. Alongside other processes such as training and testing set generation, this can lead to a sizable decrease in usable observations.

3.4 Feature Selection

3.4.1 Granger Causality

By definition, Granger causality tests the predictive power of a variable's past values on the current value of the target. Informally, the test structure can be denoted as follows:

$$H_0 : X_{t-i} \underset{\text{granger-causes}}{\not\rightarrow} y$$
$$H_1 : X_{t-i} \underset{\text{granger-causes}}{\rightarrow} y$$

GCTs can take bivariate and multivariate forms; however, MacroSim strictly uses the bivariate form. Multivariate GCTs deem if a pair of variables Granger cause the target, but present no further information on the significance of the individual variables. This structure necessitates making assumptions to assess and score each variable.

Multiple test statistics can be used to determine the significance of the relationship, such as the F-statistic, Chi-squared statistic, and the likelihood ratio. MacroSim adopts the F-statistic and performs local causality tests over slices of the data. The slices are sized to cover an 8-year period, and the `max_lag` parameter is dynamically set through the formula:

$$max\_lag = \left\lfloor \frac{N_{obs}}{20 \cdot freq} \cdot \ln(N_{obs}) \right\rceil$$

This formula provides an almost linear increase in lag size for larger datasets, while smaller datasets receive more conservative lag sizes.

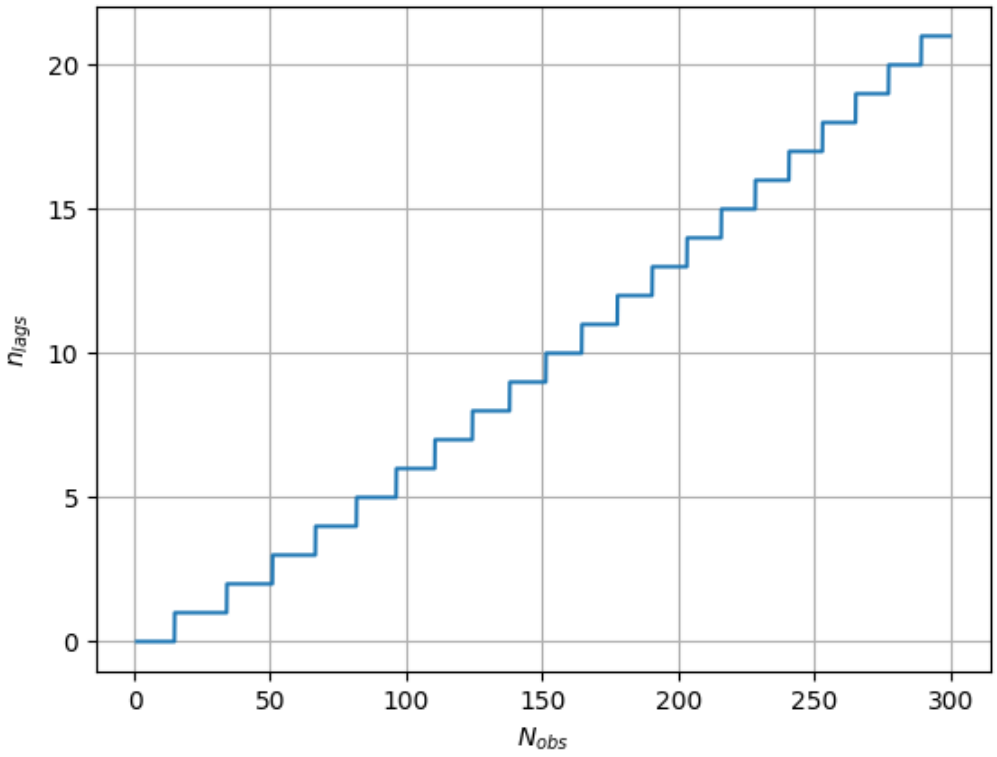

Figure 1: Function output for quarterly frequency up to 300 observations.

3.4.2 Stationarity Tests

Stationary series are more likely to be robust and reliable features due to their consistent statistical properties over time. Features with non-stationary behavior can exhibit different trends at test time, compared to the training time. Although shuffling and batching can combat this by incorporating a random sample of observations into training, MacroSim offers the option to filter out non-stationary features.

MacroSim takes a simple approach to defining and executing stationarity tests. An ADF test is performed on each feature, for lags up to `n_lags` $= \lceil \sqrt{N_{obs}} \rceil$. Macroeconomic variables tend to have trends that evolve over time, directly contradicting the assumptions of stationarity. Therefore, the ADF tests are computed for $\alpha = 0.2$. Despite the high

significance level, and the correspondingly large critical region, it is very possible that the majority of the features will be non-stationary. MacroSim implements a warning to inform users of this and encourages testing/caution before asserting stationarity in the feature space.

3.5 Equation Discovery

3.5.1 Equation Components

Although the equation discovery is ultimately handled through SR, it is important to know and navigate the parameters and components of the search space to limit the scope of the search. SR is infamously prone to overfitting due to its evolutionary search algorithms. It has the ability to add, remove, and modify terms or constants to precisely fit the curve which is very powerful compared to more static regression methods. The models are not meant to run at default parameters in most cases, and incorporating domain knowledge into the configuration is essential to avoid overfitting and ensure the model's robustness.

MacroSim uses `PySRRegressor` of the `pysr` library to perform symbolic regression. All `PySRRegressor` parameters are directly accessible to users and some defaults[2] are modified to better suit the macroeconomic context. The model receives a `TemplateExpressionSpec` as a parameter that defines the structure of function inputs, and can restrict the shapes the function can take.

The model has access to $\sin(x)$, $\cos(x)$, $\tan(x)$, e^x, $\sqrt{x}$, and $\ln(x)$ as unary operators, while basic arithmetic and exponentiation are available as binary operators. The maximum allowed complexity of the variable x can be constrained for each of these operators individually.

3.5.2 Evolutionary Search

The search algorithm implemented in `pysr` is simple in its fundamentals. This allows users to have a general idea on where the search will start and how its first few evolutionary steps are likely to progress. For illustration, in all configurations, the search starts with the function $f(X) = X$ where X is any of the available features. In most model configurations, early evolutions lead toward a simple linear form $f(X) = aX + b$ and the model continues to build more complex expressions.

For MacroSim's purposes, the initial expression being $f(X) = X$ is very important as it guarantees that the `PySRRegressor` will take at least `n_iterations` steps on this simple form, before adding an evolutionary component. Having a template expression containing the AR process results in the initial expression $f(AR(X)) = AR(X)$ allows the model

[2]No parameters that change the model weights, or evolution behavior are adjusted. Changes were made to parameters concerning complexity constrataints, allowed operations, model size, and iteration limits.

to exclusively focus on the weights during these initial steps and the rest of the search essentially serves the purpose of a secondary model fitted to the residuals of the AR process.

3.6 Recursive Simulation

...

4 Library Specifications

4.1 Core Dependencies

...

4.2 Module Structure

...

4.3 Usage

...

5 Limitations

5.1 Variable Suitability

...

5.2 Model Complexity

...

5.3 Computational Resources

...

6 Appendix

6.1 Dependency Table

Library Function	Package Dependency
`SeriesAccessor`*	`fredapi, dotenv, pandas`
`Causality`	`scikit-learn, numpy, pandas`
`Stationarity`	`scikit-learn, pandas`
`AutoReg`	`pysr, numpy, scikit-learn, pandas, tesorboard`**

* The module itself is for optional use. If used, the dependencies are required.

** The `tensorboard` dependency is only required if the user wants to use the tensorboard logging functionality of `AutoReg`. It is not required for the core functionality of the module.